

Namespace

命名空间

概述

C++ 的 **命名空间** 机制可以用来解决复杂项目中名字冲突的问题。

举个例子：C++ 标准库的所有内容均定义在 `std` 命名空间中，如果你定义了一个叫 `cin` 的变量，则可以通过 `cin` 来访问你定义的 `cin` 变量，通过 `std::cin` 访问标准库的 `cin` 对象，而不用担心产生冲突。

声明

下面的代码声明了一个名字叫 `A` 的命名空间：

```
1 namespace A {
2 int cnt;
3
4 void f(int x) { cnt = x; }
5 } // namespace A
```

声明之后，在这个命名空间外部，你可以通过 `A::f(x)` 来访问命名空间 `A` 内部的 `f` 函数。

命名空间的声明是可以嵌套的，因此下面这段代码也是允许的：

```
1 namespace A {
2 namespace B {
3 void f() { ... }
4 } // namespace B
5
6 void f() {
7     B::f(); // 实际访问的是 A::B::f(), 由于当前位于命名空间 A
8             // 内, 所以可以省略前面的 A::
9 }
10 } // namespace A
11
12 void f() // 这里定义的是全局命名空间的 f 函数, 与 A::f 和 A::B::f
13         // 都不会产生冲突
14 {
15     A::f();
16     A::B::f();
17 }
```

using 指令

声明了命名空间之后，如果在命名空间外部访问命名空间内部的成员，需要在成员名前面加上 `命名空间::`。

有没有什么比较方便的方法能让我们直接通过成员名访问命名空间内的成员呢？答案是肯定的。我们可以使用 `using` 指令。

`using` 指令有如下两种形式：

1. `using 命名空间::成员名;`：这条指令可以让我们省略某个成员名前的命名空间，直接通过成员名访问成员，相当于

将这个成员导入了当前的作用域。

2. `using namespace 命名空间;`：这条指令可以直接通过成员名访问命名空间中的 **任何** 成员，相当于将这个命名空间的所有成员导入了当前的作用域。

因此，如果执行了 `using namespace std;`，就会在当前作用域将 `std` 中的所有名字引入到全局命名空间当中。这样，我们就可以用 `cin` 代替 `std::cin`，用 `cout` 代替 `std::cout`。

► `using` 指令可能会导致命名冲突！

有了 `using` 指令，[C++ 语法基础](#) 中的代码可以有这两种等价写法：

```
1 #include <iostream>
2
3 using std::cin;
4 using std::cout;
5 using std::endl;
6
7 int main() {
8     int x, y;
9     cin >> x >> y;
10    cout << y << endl << x;
11    return 0;
12 }
```

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     int x, y;
7     cin >> x >> y;
8     cout << y << endl << x;
9     return 0;
10 }
```

无名命名空间

当我们在一个作用域里只定义了一个用于防止名字冲突的命名空间时，其定义和使用将可以变得非常简洁。我们可以使用无名命名空间。

形如 `namespace { /* something ... */ }`（省略命名空间的名字）定义的命名空间被称为无名命名空间。一个文件里的无名命名空间会被视为拥有独有的名字，和其他命名空间都不同，但同一个作用域内多个无名命名空间被视为相同的命名空间。在无名命名空间定义后，其中的名字在其外的作用域内可以在使用时被查找找到，如同在无名命名空间定义后加入了一条 `using namespace` 指令。

应用

防止子任务间名字冲突

在一些具有多个子任务的问题中，我们可以对每个子任务各定义一个命名空间，在其中定义我们解决该子任务所需要的变量与函数，这样即使两个子任务的实现中即使声明了相同名字也不会冲突，从而使各个子任务间互不干扰，会在一定程度上方便调试，也会改善程序的可读性。

防止与标准库以及环境引入的名字冲突

同时，使用命名空间也可以防止一些算法竞赛中常用的名字与标准冲突，如下例：

```
1 #include <cmath>
2
3 #include <vector>
4 using namespace std;
5
6 namespace Sol {
7 int end; // std::end 被 using namespace std; 引入
8
9 int y1; // y1 是 POSIX 定义的第二类 Bessel 函数
10
11 // 因此通常情况下, 在 Linux 下会有冲突而在 Windows 下没有
12
13 void solve() {
14 // 在 Sol::solve() 里无限定 (不用 ::) 地使用我们声明的 end 以及 y1
15 // 并不会导致名字冲突; 而若以上代码在全局命名空间中, 将会导致冲突: 其中 end
16 // 只在名字查找 (即编译使用它的代码) 时与 std::end 冲突, 而 y1
17 // 在声明时就会冲突; 并且 y1 的冲突因为与环境有关甚至在 Windows
18 // 下不会被发现, 却会在 Linux 的评测环境下造成编译错误。
19 }
20 } // namespace Sol
21
22 int main() { Sol::solve(); }
```

参考

- [Namespaces - cppreference.com](https://en.cppreference.com/namespaces)

本页面最近更新: 2025/4/11 22:52:14, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [billchenchina](#), [Chrogeek](#), [countercurrent-time](#), [Enter-tainer](#), [H-J-Granger](#), [lr1d](#), [ksyx](#), [NachtgeistW](#), [ntt998244353](#), [ouuan](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用